

Insight Ladder — Day 4

How far apart can you place them?

Priyanka | September 24, 2026 | Medium relative to this week | 120 minutes

Main problem

Choose exactly k positions from n distinct integer coordinates to maximize the minimum distance between any two chosen positions. Input is unsorted; $2 \leq k \leq n \leq 200,000$ and coordinates lie between $-1,000,000,000$ and $1,000,000,000$. Return the optimal integer distance. Target $O(n \log n + n \log(W+1))$ time, where $W = \max(x) - \min(x)$; sorting a copy is allowed. Use wide subtraction in C.

Examples and checks: $[1, 2, 4, 8, 9]$, $k=3$ gives 3. $[0, 10]$, $k=2$ gives 10. When $k=n$, test against the smallest adjacent gap after sorting. Include negative coordinates.

Optional variation

Now two specified coordinates must be selected. No new implementation is required: give a small counterexample showing why the original greedy checker cannot simply ignore the mandatory positions, or why checking whether its one chosen set includes both positions can wrongly reject a feasible distance. State precisely what your example disproves.

Before reading hints: What evidence makes a proposed distance feasible? Why is placing a selected point as early as possible safe in the original problem? Does a larger candidate distance become easier or harder?

Your 120 minutes

- 0–10: closed-notes recall of one invariant or failed assumption from the previous session.
- 10–100: solve the main problem. Write a direct solution, identify repeated work, state a claim, try to break it, then implement. Attempt the optional variation only when the main solution is secure; it stays inside these 90 minutes.
- 100–120: stop coding, explain correctness and cost, review evidence, and log partial work. A full solution read during review does not turn an unfinished timed attempt into a solve.
- Use C11 OR Python 3 consistently. After 15 minutes without a useful new step, record the blocker and request one hint or continue. Record the time and level of every hint or appendix consultation.

Submit your evidence

Submit code, tests, the key claim and its justification, runtime/space, minutes spent, and hints used. Main status: independent solve / assisted solve / partial / unattempted. Insight: independent / after small hint / after method supplied. Track variation coding or reasoning separately; a counterexample is not another code solve. Count a main solve only when working code meets the contract and you can justify it. There is no completion quota.

Solution appendix

Open only after an attempt; record what you read and when. Read one hint at a time. The explanation below supplies the method and cannot count as independent discovery.

Progressive hints

1. For a proposed distance d , try to decide only whether k positions can be chosen.
2. After sorting, choose the first coordinate and then each earliest coordinate at least d beyond the last chosen one. Compare these choices with those of any feasible placement.
3. Feasibility is true for smaller d and false for larger d . Search for the last feasible distance; use a midpoint that guarantees progress.

Main solution reasoning

Sort coordinates. For a candidate d , choose the first position, then greedily the earliest position $\geq \text{last} + d$ until k are chosen or positions run out. Prove by induction that the greedy j th choice is no later than the j th choice of any feasible placement: this is true for $j=1$, and that placement's next choice is also far enough beyond the earlier greedy choice. Greedy therefore leaves at least as much room for future choices. Feasibility decreases as d increases. Search $[0, W]$ with upper midpoint $\text{mid} = \text{lo} + (\text{hi} - \text{lo} + 1) // 2$; feasible implies $\text{lo} = \text{mid}$, otherwise $\text{hi} = \text{mid} - 1$. The upper midpoint avoids a stall on a two-value interval. Return lo . It suffices to constrain adjacent chosen gaps since nonadjacent distances are sums of positive adjacent gaps. Each check is $O(n)$; total is $O(n \log n + n \log(W+1))$, with sorting storage reported separately.

Variation review

False acceptance: coordinates $[0, 4, 5, 9]$, $k=3$, $d=4$, mandatory 4 and 5. The unconstrained checker accepts $[0, 4, 9]$, but no valid placement can contain both mandatory points because their gap is 1. False rejection by post-checking one greedy output: $[0, 3, 6, 9]$, $k=3$, $d=3$, mandatory 3 and 9. Greedy stopping at three picks returns $[0, 3, 6]$; rejecting it misses the feasible set $[0, 3, 9]$. The original proof allows replacing choices with earlier ones; mandatory choices restrict that freedom.

Exit ticket and repair

What evidence makes a proposed distance feasible? Why is placing a selected point as early as possible safe in the original problem? Does a larger candidate distance become easier or harder?

If the explanation is still unclear: Write down the direction of feasibility and trace the last-true search on an interval containing two integers.

Log: main attempted __; independent solve __; assisted solve __; partial __; variation status __; insight assistance __; minutes to first useful claim __; total minutes __.